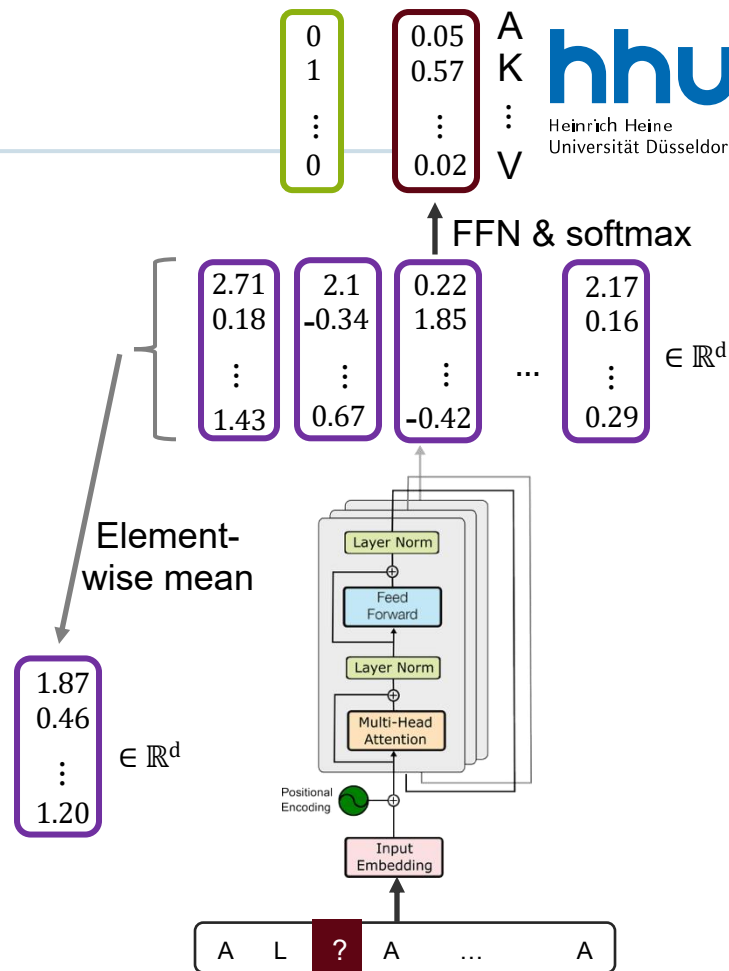


Training Transformer Networks

Supervised Fine-Tuning

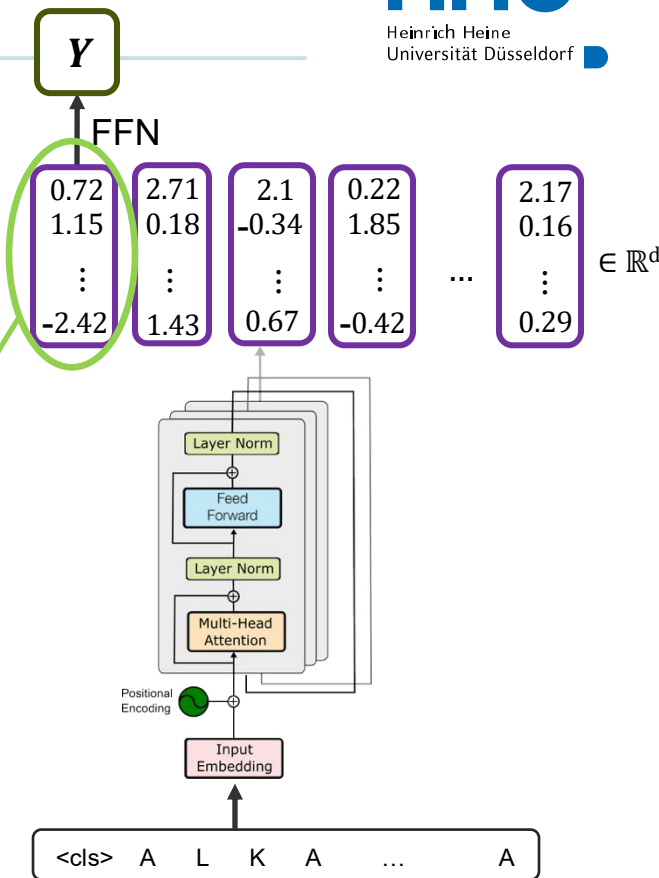
Self-Supervised Training

- Advantage:
 - Meaningful protein representations without labeled training data
- Disadvantages:
 - We do not train the model for a specific task and cannot be sure that it extracts all relevant information
 - Taking the mean over many amino acid representations results in information loss
- How can we overcome the disadvantages?
 - We can train an encoder for a specific task and force it to store all task-relevant protein information in a single vector



Supervised Training

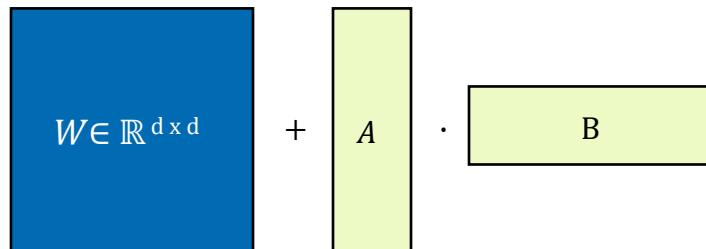
- We add a token (<cls>) at the beginning of the sentence
 - We want to use the representation of this token as a whole protein representation
- Training task: predict protein property using updated <cls>-token representation
- Training end-to-end (encoder & FFN) forces the model to store all task-relevant information in the updated <cls> representation
- After training (two options):
 - Use this model as our final prediction model
 - Extract updated <cls> representation as a task-specific protein representation and use as input for another model
- We start this training phase (fine-tuning) by using the parameters of the pre-trained Transformer Network for the mask token prediction



Alternative Supervised Training Techniques

■ LoRA (Low-Rank Adaptation) Finetuning

- Instead of fine-tuning the parameters W in the attention functions directly, we can replace $W \in \mathbb{R}^{d \times d}$ by:
 - $W' = W + \Delta W = W + A \cdot B$ with $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times d}$ with $r \ll d$
 - A and B are trainable, while W stays frozen



<u>Component</u>	<u>Status</u>
Original model weights	Frozen
LoRA adapters (A & B)	Trainable
Prediction head	Trainable
LayerNorm / embeddings	Usually frozen
FFN layers	Usually frozen, but optional LoRA

- Example $d = 1024, r = 8$
 - Full fine-tuning: $1024 \times 1024 = 1,048,576$ parameters to finetune
 - LoRA finetuning:
 - A and B have each $1024 \times 8 = 8192$ parameters
 - 98% fewer parameters

<u>Advantages</u>	<u>Disadvantages</u>
Drastically fewer trainable parameters (requires much less GPU memory)	May underperform compared to full fine-tuning on some tasks / Less flexible
Avoids overwriting pretrained weights	Needs tuning of extra hyperparameters (e.g. rank r)
Faster training	Slightly more complex architecture and inference logic